Rogue Agent: The Economics of Compute in Agentic Age

Working Paper

September 2026

Abstract

Firms adopting AI for software development must trade off engineering labor against increasing compute requirements. This paper develops a task-based model in which heterogeneous software tasks are allocated across human, reasoning-assisted, and agentic workflows. The firm governs compute use through either an internal price or an aggregate quota. The model shows that when agentic compute costs grow faster than labor savings, agentic coding is eventually excluded for sufficiently complex tasks. Under additional restrictions, optimal agentic usage is concentrated within an intermediate or low complexity region.An engineer effort and AI depth problem is solved to estimate modality specific labor and compute requirements, then integrates those responses over empirical task-complexity distributions from public Jira repositories. An application to Red Hat shows diminishing marginal returns to compute. Initial compute capacity shifts tasks from unaided human production to reasoning-assisted workflows, while additional capacity expands agentic coding. Even when the compute constraint ceases to bind, reasoning remains optimal for a subset of tasks. The results highlight task composition and internal compute governance as central determinants of organizational AI adoption.

1 Introduction

According to Zebracat, 72% of companies report using AI tools in at least one of their departments. And 65% of employees say they have access to at least one AI tool to support their daily work.

How does a firm allocate heterogeneous tasks across human, reasoning assisted, and agentic workflows when engineering capacity is scarce and agentic compute costs increase sharply with task complexity?

This paper studies the allocation of agentic AI across task based problems under alternative compute-governance regimes. The central result is while compute resource scarcity explains why firms ration agentic workflows, the task-level comparative advantage explains why reasoning survives even after that scarcity is removed.

1.1 The Production Menu

Based on recent empirical results (Bai et al. 2026), we model the structural costs of the modalities as follows, where D is the complexity of any given tasks, and $C_m(D)$ is the cost of completing a task with a complexity of :

- **Code Chat ($m = C$):** Requires high human effort but minimal API overhead.

$$L_C(D) = \alpha_C D$$
$$C_C(D) = \theta_C D$$

- **Code Reasoning ($m = R$):** Reduces human effort but introduces polynomial reasoning tokens.

$$L_R(D) = \alpha_R D$$
$$C_R(D) = \theta_R D^2$$

- **Agentic Coding ($m = A$):** Autonomous loops minimize human typing but accumulate expensive context windows, resulting in exponential token scaling.

$$L_A(D) = \alpha_A D$$
$$C_A(D) = \theta_A e^{\beta D}$$

Therefore, the baseline compute technology is

$$\begin{aligned} C_C(D) &= \theta_C D, \\ C_R(D) &= \theta_R D^2, \\ C_A(D) &= \theta_A \exp(\beta D), \end{aligned} \tag{1}$$

Where $\alpha_A < \alpha_R < \alpha_C$ (agents save human labor), and $\beta > 0$ dictates the context-window increase of the ReAct architecture. The linear and quadratic forms provide tractable benchmarks for chat and reasoning workflows. The exponential agentic form is a reduced-form approximation to context accumulation, repeated tool use, and failure-induced retries.

The true compute cost in (1) is expected monetary cost, not merely the price of one model call. A more detailed measurement equation decomposes it into token categories:

$$C_{imr} = 10^{-6} \left(p_m^{\text{in}} Q_{imr}^{\text{in}} + p_m^{\text{out}} Q_{imr}^{\text{out}} + p_m^{\text{write}} Q_{imr}^{\text{write}} + p_m^{\text{read}} Q_{imr}^{\text{read}} \right), \tag{2}$$

where i indexes tasks, m models or modalities, and r repeated runs. This decomposition is motivated by evidence that agentic costs are dominated by accumulated input context rather than generated output (Bai et al. 2026). Equation (1) is the reduced-form conditional mean of (2).

Our exponential specification can be interpreted through a failure hazard model of agentic production. Kwa et al. (2026) document a strong negative relationship between AI-agent success and the human time required to complete a task. Furthermore, Ord (2025) shows that success rate of tasks is consistent with an approximately constant failure hazard per unit of task length. If t denotes task

length, this mechanism implies

$$p_A(t) = \exp(-\lambda_A t).$$

For a workflow that repeats approximately independent attempts until successful completion, the expected number of attempts is therefore $1/p_A(t) = \exp(\lambda_A t)$.

Let $c_A^{\text{run}}(t)$ denote the compute used by one agentic trajectory. Expected compute per successful completion is

$$E[C_A(t)] = \frac{c_A^{\text{run}}(t)}{p_A(t)} = c_A^{\text{run}}(t) \exp(\lambda_A t).$$

Cumulative context processing may cause within-run cost to grow polynomially in trajectory length, while the declining completion probability introduces an exponential reliability multiplier. If human task duration is approximately affine in the paper's complexity measure, the resulting expected completion cost is a polynomial multiplied by an exponential function of complexity. We use a pure exponential as a parsimonious reduced form approximation to this relationship.

1.2 The Engineer's Problem

Before analyzing company level portfolio allocation, we first formalize the individual engineer's dynamic decision-making process over a continuous planning horizon T. Let X_t represent cumulative task progress at time t. Progress is driven by human effort e_t and the reasoning depth of deployed AI agents k_t:

$$dX_t = (e_t + \gamma k_t^{\alpha})\, dt, \quad X_0 = 0 \tag{3}$$

where $\gamma > 0$ is AI productivity and $0 < \alpha < 1$ represents diminishing returns to agentic depth on a single task. The token cost of the agent scales exponentially with reasoning depth due to context window accumulation: $C(k_t) = \theta e^{\beta k_t}$ and an increased number of attempts.

Think of $C(\cdot)$ not as the cost of a single perfect run, but as the Expected Cost of completing the task. Therefore, Expected Cost = (Cost per Attempt) $\times$ (Expected Number of Attempts). As a task gets harder, two things happen simultaneously:

Cost per attempt increases: The agent has to read more files and write more code, naturally inflating the context window.

Number of attempts increases: The agent's accuracy drops, meaning it fails and triggers retry loops.

Because both the size of the payload and the number of times it must be sent are growing, they multiply together. Therefore, the single exponential term $e^{\beta k_t}$ captures this compound effect without needing a multi-variable probability expression.

The engineer maximizes lifetime utility, balancing output value β_y, human disutility of effort η, and a corporate compute tax τ_t:

$$\max_{\{e_t, k_t\}} \mathbb{E}\left[\int_0^T e^{-\rho t}\left(\beta_y X_t - \frac{\eta}{2} e_t^2 - \tau_t \theta e^{\beta k_t}\right) dt + e^{-\rho T} S(X_T)\right] \tag{4}$$

The current-value Hamiltonian is defined as:

$$\mathcal{H} = \beta_y X_t - \frac{\eta}{2} e_t^2 - \tau_t \theta e^{\beta k_t} + \lambda_t \left(e_t + \gamma k_t^{\alpha} \right) \tag{5}$$

Applying the First-Order Conditions with respect to the controls e_t and k_t yields:

$$e_t^* = \frac{\lambda_t}{\eta} \tag{6}$$

$$\lambda_t \gamma \alpha k_t^{\alpha - 1} = \tau_t \theta \beta e^{\beta k_t} \tag{7}$$

Equation 7 defines the optimal agentic depth k_t^*. If the firm imposes no tax ($\tau_t \to 0$), the right side vanishes, and the engineer rationally picks $k_t^* \to \infty$, leading to runaway exponential token usage. A strictly positive tax $\tau_t > 0$ forces an internal boundary on k_t^*. However, continuously throttling the depth k_t on a single task is often operationally intractable. In practice, engineers face a discrete choice of AI modalities, which we formalize in the following sprint portfolio model.

2 Background and Model

This paper presents a variant of Acemoglu–Restrepo's task-based automation model (2019), where tasks are indexed by complexity and assigned to whichever factor is cheaper at the margin. This is a threshold model where the automation cost function is exponential in complexity rather than monotonically falling, which changes the comparative statics in a way that deviates from the standard results. The central economic object is therefore not an aggregate AI productivity shifter, but the modality assigned to each task. This distinction is particularly appropriate for software engineering, where AI systems interact with repositories, coding tools, testings, and review processes. OpenHands provides an example of this agent-environment architecture, allowing agents to write code, execute commands, browse, and interact with sandboxed environments (Wang et al. 2025). Recent trajectory-level evidence further indicates that agentic coding is unusually input intensive and highly variable across runs, and only imperfectly predicted by task difficulty rated by human (Bai et al. 2026).

2.1 Modality-Contingent Engineer Problem

As descried above, the company can make available one of three workflow modalities,

$$m \in \mathcal{M} = \{C, R, A\}, \tag{8}$$

Conditional on modality m and task complexity D, the engineer chooses human effort e_t and AI reasoning depth k_t over a finite horizon $t \in [0, T]$. Task progress evolves according to

$$\dot{X}_t = e_t + \gamma_m k_t^{\varphi_m}, \qquad X_0 = 0, \tag{9}$$

where $\gamma_m \geq 0$ measures the productivity of AI reasoning under modality m, and $\varphi_m \in (0,1)$ allows diminishing returns to AI depth.

Human effort directly increases progress. AI reasoning depth contributes additional progress, but its marginal product declines when $\varphi_m < 1$. The modality affects both AI productivity and the set of reasoning depths available to the engineer:

$$k_t \in [\underline{k}_m, \overline{k}_m]. \tag{10}$$

For the human-oriented modality, we impose $\underline{k}_C = \overline{k}_C = 0$ and $\gamma_C = 0$. Reasoning and agentic modalities permit progressively larger AI-depth choices.

The cost of compute consumption is

$$c_m(k_t) = \theta_m \exp(\beta_m k_t), \tag{11}$$

where $\theta_m \geq 0$ is the modality-specific compute scale and $\beta_m > 0$ governs the rate at which compute requirements increase with reasoning depth.

The exponential specification captures the idea that deeper AI workflows require longer contexts, more model calls, additional tool interactions, and potentially more retries. Existing evidence finds that agentic coding uses substantially more tokens than conversational or code-reasoning workflows, that input tokens are a major source of cost, and that repeated runs of the same task can exhibit considerable cost variation (Bai et al. 2026). The deterministic specification in Equation (11) should therefore be interpreted as expected compute use in the baseline model rather than as a complete description of realized token consumption.

The engineer solves

$$V_m^E(D) = \max_{\{e_t, k_t\}_{t=0}^T} \left\{ \int_0^T e^{-\rho t} \left[bX_t - \frac{\eta}{2} e_t^2 - \tau c_m(k_t) \right] dt + e^{-\rho T} S(X_T; q(D)) \right\}, \tag{12}$$

subject to Equations (9) and (10), together with

$$e_t \in [\underline{e}, \overline{e}]. \tag{13}$$

The parameter $b \geq 0$ represents the engineer's reduced-form incentive to generate progress. It can reflect performance evaluation, deadlines, career concerns, or the value of completing assigned work. It is not the company's revenue per unit of output. The parameter $\eta > 0$ governs the convex disutility of human effort. The parameter $\tau \geq 0$ is the extent to which the engineer internalizes compute use, for example through an internal token price, usage limit, or organizational chargeback system.

The baseline terminal payoff is

$$S(X_T; q(D)) = vq(D) - \frac{\kappa}{2}\left[q(D) - X_T\right]^2, \tag{14}$$

where $\kappa > 0$ penalizes failure to reach the task target. The term $vq(D)$ is constant conditional on D and therefore does not affect the engineer's controls. It allows the value function to include the gross value associated with the assigned task, while the quadratic term provides the operative completion incentive.

A high value of κ approximates a completion requirement without imposing a numerically discontinuous hard constraint $X_T \geq q(D)$. Because the penalty is finite, the engineer may optimally leave a small shortfall when the marginal cost of additional effort or compute exceeds the marginal completion benefit. The numerical implementation reports this shortfall explicitly.

2.2 Continuum Approximation

The finite-sprint allocation problem is a multiple choice knapsack problem. Each task must be assigned to one modality, while aggregate compute consumption cannot exceed the firm's budget. This finite problem is the appropriate formulation for numerical implementation, but its integer structure makes generalized analytical solution difficult.

To obtain a tractable population counterpart, let task complexity be distributed according to a cumulative distribution function F on support $\mathcal{D} \subseteq [0, \infty)$. The firm chooses a measurable modality policy

$$\pi : \mathcal{D} \to \mathcal{M}, \qquad \mathcal{M} = \{C, R, A\}. \tag{15}$$

Let b denote the available compute budget per task. If the firm has N tasks and total compute budget B, then $b = B/N$. The continuum allocation problem is

$$\begin{aligned} V(b) = \min_{\pi} \quad & \int_{\mathcal{D}} wL_{\pi(D)}(D)\, dF(D) \\ \text{subject to} \quad & \int_{\mathcal{D}} C_{\pi(D)}(D)\, dF(D) \leq b. \end{aligned} \tag{16}$$

Equation (16) replaces the finite collection of tasks with the distribution of task complexity. This is useful for the present application because organizations differ not only in the number of tasks they face, but also in the shape of their complexity distributions.

The continuum formulation should be interpreted as a population approximation to the finite integer problem. When F has a mass at a particular complexity value, the continuous formulation allows the firm to assign fractions of same-complexity tasks to different modalities. In an actual finite sprint, such fractions correspond to assigning different tasks within that complexity category to different modalities. Exact equivalence between the continuum and integer problems need not hold for every finite budget because tasks are indivisible.

Let $\lambda \geq 0$ denote the Lagrange multiplier on the expected compute constraint. The Lagrangian is

$$\mathcal{L}(\pi, \lambda) = \int_{\mathcal{D}} \left[wL_{\pi(D)}(D) + \lambda C_{\pi(D)}(D) \right] dF(D) - \lambda b. \tag{17}$$

Conditional on λ, the integrand is separable across task complexity. Hence, under the regularity conditions stated in Section 3, the optimal policy satisfies

$$\pi^*_\lambda(D) \in \arg\min_{m \in \mathcal{M}} \{ wL_m(D) + \lambda C_m(D) \} \tag{18}$$

for F-almost every D.

Equation (18) is the analytical simplification generated by the continuum formulation. The shape of the resulting policy depends on the pairwise differences in labor and compute requirements. As shown below, exponential growth in agentic compute guarantees that agentic coding is eventually excluded at sufficiently high complexity. A single upper threshold requires an additional shape condition.

3 Theoretical Results

This section characterizes task allocation under an aggregate compute constraint. The main result is an eventual exclusion theorem: because agentic compute grows exponentially while its labor advantage grows at most linearly, agentic coding is not optimal for sufficiently complex tasks whenever compute has a strictly positive shadow price.

A stronger single threshold result requires an additional shape restriction. In particular, exponential tail growth does not imply that agentic coding is optimal for every low-complexity task. A positive fixed agentic compute cost can instead generate an intermediate region in which agentic coding is optimal.

3.1 Expected Allocation Problem

Let task complexity be distributed according to the cumulative distribution function F on support $\mathcal{D} \subseteq [0, \infty)$. The firm chooses a measurable modality policy

$$\pi : \mathcal{D} \to \mathcal{M}, \qquad \mathcal{M} = \{C, R, A\}.$$

Let b denote the available compute budget per task. The continuum allocation problem is

$$\begin{aligned} V(b) = \min_{\pi} \quad & \int_{\mathcal{D}} wL_{\pi(D)}(D)\, dF(D) \\ \text{subject to} \quad & \int_{\mathcal{D}} C_{\pi(D)}(D)\, dF(D) \leq b. \end{aligned} \tag{19}$$

If the firm faces N tasks and a total budget B, then $b = B/N$. The expected-value formulation is the continuum analogue of the finite multiple-choice knapsack problem. When F contains atoms, the convexified problem permits the firm to assign fractions of same-complexity tasks to different modalities. In the finite integer problem, exact price–quota equivalence need not hold because tasks are indivisible.

We impose the following regularity conditions.

For every modality $m \in \mathcal{M}$, $L_m(D)$ and $C_m(D)$ are measurable and integrable under F. If $\mathcal{D}$ is unbounded, then

$$\int_{\mathcal{D}} e^{\beta D}\, dF(D) < \infty.$$

The convexified allocation problem is feasible and satisfies a constraint qualification. When the quota binds, its multiplier satisfies $\lambda_b > 0$.

The exponential-moment restriction is automatically satisfied when task complexity has bounded support, as in the empirical implementation.

3.2 Pointwise Decentralization

The Lagrangian associated with Equation (19) is

$$\mathcal{L}(\pi, \lambda) = \int_{\mathcal{D}} \left[wL_{\pi(D)}(D) + \lambda C_{\pi(D)}(D) \right] dF(D) - \lambda b, \qquad \lambda \geq 0. \tag{20}$$

Conditional on a multiplier λ, the Lagrangian is separable across task complexity.

Proposition 1 (Pointwise allocation)**.** *Suppose Assumption 3.1 holds. Any solution to the convexified quota problem can be decentralized by a multiplier $\lambda_b \geq 0$ such that, for F-almost every D,*

$$\pi_b^*(D) \in \arg\min_{m \in \mathcal{M}} \left\{ wL_m(D) + \lambda_b C_m(D) \right\}. \tag{21}$$

If the budget binds and is locally valuable, then $\lambda_b > 0$.

Proof. Under the stated regularity and constraint-qualification conditions, the convexified problem satisfies the usual Lagrangian optimality conditions. For a fixed multiplier, the integrand in Equation (20) contains no cross-task term. The minimizing policy therefore selects a cost-minimizing modality pointwise for F-almost every D. Complementary slackness gives

$$\lambda_b \left[\int C_{\pi_b^*(D)}(D)\, dF(D) - b \right] = 0.$$

Local value of additional compute implies $\lambda_b > 0$ when the constraint binds. □

Proposition 1 establishes the connection between the quota and internal-price regimes. An exogenous internal compute price τ produces the same pointwise rule as Equation (21) after setting $\tau = \lambda_b$. The

difference is that τ is selected directly under the price regime, whereas λ_b is determined endogenously by the aggregate budget.

3.3 Agentic versus Reasoning

To characterize the agentic boundary, consider the comparison between reasoning and agentic coding. The baseline resource requirements are

$$L_R(D) = \alpha_R D, \qquad L_A(D) = \alpha_A D, \tag{22}$$

$$C_R(D) = \theta_R D^2, \qquad C_A(D) = \theta_A e^{\beta D}, \tag{23}$$

where

$$\alpha_R > \alpha_A \geq 0, \qquad \theta_R > 0, \qquad \theta_A > 0, \qquad \beta > 0.$$

Define the labor saving from assigning a task to agentic rather than reasoning coding:

$$\Delta L(D) = L_R(D) - L_A(D) = (\alpha_R - \alpha_A)D. \tag{24}$$

Similarly, define incremental agentic compute:

$$\Delta C(D) = C_A(D) - C_R(D) = \theta_A e^{\beta D} - \theta_R D^2. \tag{25}$$

For a fixed shadow price $\lambda > 0$, define the net advantage of agentic coding relative to reasoning:

$$G_{AR}(D; \lambda) = w\Delta L(D) - \lambda \Delta C(D). \tag{26}$$

Agentic coding is weakly preferred to reasoning if and only if

$$G_{AR}(D; \lambda) \geq 0.$$

When $\Delta C(D) > 0$, it is useful to define the marginal labor saving per additional unit of compute:

$$R_{AR}(D) = \frac{w\Delta L(D)}{\Delta C(D)} = \frac{w(\alpha_R - \alpha_A)D}{\theta_A e^{\beta D} - \theta_R D^2}. \tag{27}$$

The notation R_{AR}, rather than ROI, emphasizes that this object is a resource tradeoff rather than a conventional financial return on investment. Where $\Delta C(D) > 0$, agentic coding is preferred when

$$R_{AR}(D) \geq \lambda.$$

3.4 Eventual Exclusion

Theorem 2 (Eventual exclusion of agentic coding)**.** *Suppose Equations* (22) *and* (23) *hold and* $\lambda > 0$*. Then there exists a finite value* $D_H(\lambda)$ *such that*

$$G_{AR}(D;\lambda) < 0$$

for every $D > D_H(\lambda)$*. Consequently, agentic coding is not optimal for sufficiently complex tasks. In the three-modality model, any such task is assigned to either* C *or* R*.*

Proof. Substituting the baseline resource functions into Equation (26) gives

$$G_{AR}(D;\lambda) = w(\alpha_R - \alpha_A)D - \lambda\theta_A e^{\beta D} + \lambda\theta_R D^2. \tag{28}$$

The positive terms in Equation (28) grow linearly and quadratically in D, while the negative term grows exponentially. Therefore,

$$\lim_{D\to\infty} G_{AR}(D;\lambda) = -\infty. \tag{29}$$

By the definition of convergence to negative infinity, there exists a finite $D_H(\lambda)$ such that $G_{AR}(D;\lambda) < 0$ for all $D > D_H(\lambda)$. Since reasoning then has strictly lower shadow-cost-adjusted resource cost than agentic coding, agentic coding cannot be globally optimal among $\{C, R, A\}$. □

Theorem 2 establishes a robust upper-tail result. It does not imply that every task below $D_H(\lambda)$ is assigned to agentic coding.

Indeed,

$$G_{AR}(0;\lambda) = -\lambda\theta_A < 0. \tag{30}$$

Thus, with a positive fixed agentic compute requirement, reasoning is also preferred in a neighborhood of zero. Agentic coding may therefore be optimal only at intermediate complexity levels.

3.5 Intermediate Agentic Region

A stronger characterization requires a shape restriction on $R_{AR}(D)$.

[Single-peaked resource return] On the region in which $\Delta C(D) > 0$, the function $R_{AR}(D)$ is continuous, first strictly increasing and then strictly decreasing, with a unique maximizer D_M.

This assumption can be checked numerically for any proposed parameterization. It should not be inferred solely from exponential tail dominance.

Proposition 3 (Intermediate agentic region)**.** *Suppose Assumption 3.5 holds and*

$$0 < \lambda < R_{AR}(D_M).$$

Then the equation

$$R_{AR}(D) = \lambda$$

has two solutions,

$$D_L(\lambda) < D_M < D_H(\lambda),$$

and agentic coding is preferred to reasoning on

$$D \in [D_L(\lambda), D_H(\lambda)].$$

If $D_L(\lambda)$ lies below the lower support point of the empirical task distribution, the observed allocation reduces to a single upper boundary:

$$A \quad \textit{for} \quad D \leq D_H(\lambda), \qquad R \quad \textit{for} \quad D > D_H(\lambda).$$

Proof. Continuity and strict single-peakedness imply that every horizontal line strictly between zero and the maximum value of R_{AR} intersects the function exactly twice. Agentic coding is preferred where $R_{AR}(D) \geq \lambda$, which is the interval between the two intersections. If the lower intersection lies below the support of observed complexity, only the upper intersection affects the empirical allocation. □

This proposition matches the numerical reference exercise. The continuous choice equation produces a lower and upper crossing, but the lower crossing lies below the observed support. The empirical policy therefore appears as a conventional upper-cutoff allocation.

3.6 Expected Modality Shares

The theoretical allocation maps into expected organizational outcomes through F. Under the intermediate-region policy, the expected agentic share is

$$s_A(\lambda) = \int_{\mathcal{D}} \mathbf{1}\{D_L(\lambda) \leq D \leq D_H(\lambda)\}\, dF(D). \tag{31}$$

If the lower crossing lies below the support of F, then

$$s_A(\lambda) = F(D_H(\lambda))\,. \tag{32}$$

When D is discrete, F is a step function. Consequently, an upper boundary equal to the median complexity need not imply an agentic share of one-half. The share equals the cumulative probability mass at and below the boundary.

Expected labor and compute are

$$\overline{L}(\lambda) = \int_{\mathcal{D}} L_{\pi_\lambda(D)}(D)\, dF(D), \tag{33}$$

$$\overline{C}(\lambda) = \int_{\mathcal{D}} C_{\pi_\lambda(D)}(D)\, dF(D). \tag{34}$$

These expected outcomes, rather than the boundary alone, are the economically relevant quantities for the firm.

3.7 Budget Comparative Statics

Suppose the lower boundary lies below the support of F, so the policy has the upper-threshold form

$$\pi(D) = \begin{cases} A, & D \le D_H, \\ R, & D > D_H. \end{cases}$$

The expected compute associated with a boundary D_H is

$$\mathcal{B}(D_H) = \int_0^{D_H} C_A(D)\, dF(D) + \int_{D_H}^{\infty} C_R(D)\, dF(D). \tag{35}$$

Similarly, expected labor is

$$\mathcal{H}(D_H) = \int_0^{D_H} L_A(D)\, dF(D) + \int_{D_H}^{\infty} L_R(D)\, dF(D). \tag{36}$$

Proposition 4 (Marginal value of compute)**.** *Suppose F has density f, $f(D_H) > 0$, $\Delta C(D_H) > 0$, and $\Delta L(D_H) > 0$. Then*

$$\mathcal{B}'(D_H) = f(D_H)\Delta C(D_H) > 0, \tag{37}$$

$$\mathcal{H}'(D_H) = -f(D_H)\Delta L(D_H) < 0. \tag{38}$$

If the quota binds so that $\mathcal{B}(D_H) = b$, then

$$\frac{dD_H}{db} = \frac{1}{f(D_H)\Delta C(D_H)} > 0, \tag{39}$$

and

$$-\frac{d[w\mathcal{H}(D_H)]}{db} = \frac{w\Delta L(D_H)}{\Delta C(D_H)} = \lambda_b. \tag{40}$$

Proof. Equations (37) and (38) follow from Leibniz's rule. The implicit function theorem applied to $\mathcal{B}(D_H) = b$ gives Equation (39). Combining the derivatives and using the marginal indifference condition gives Equation (40). □

Proposition 4 supplies the expected-value interpretation of the shadow price. At the allocation boundary, λ_b equals the marginal reduction in expected labor cost obtained from one additional unit of expected compute.

3.8 Comparative Statics of the Upper Boundary

The previous claim of invariant automation ordering is too strong. Changes in θ_A and θ_R can shift both crossings, eliminate the agentic region, or alter which modality dominates at low complexity. Nevertheless, local comparative statics can be signed at a regular upper crossing.

Proposition 5 (Upper-boundary comparative statics)**.** *Let D_H satisfy*

$$G_{AR}(D_H; \lambda) = 0$$

and suppose

$$\frac{\partial G_{AR}(D_H; \lambda)}{\partial D} < 0, \qquad \Delta C(D_H) > 0.$$

Then, holding the other parameters fixed,

$$\frac{\partial D_H}{\partial w} > 0, \qquad \frac{\partial D_H}{\partial \lambda} < 0, \tag{41}$$

$$\frac{\partial D_H}{\partial \theta_A} < 0, \qquad \frac{\partial D_H}{\partial \beta} < 0, \tag{42}$$

$$\frac{\partial D_H}{\partial \theta_R} > 0. \tag{43}$$

The signs have a direct interpretation. A higher value of engineering labor expands the agentic region, while a higher compute shadow price, agentic base cost, or context-growth rate contracts it. More expensive reasoning compute expands the region in which agentic coding is preferred.

These are local results at the descending upper crossing. They do not establish that the global allocation ordering is invariant to prices.

3.9 Tail Approximation

At sufficiently high complexity,

$$\theta_A e^{\beta D} \gg \theta_R D^2.$$

The upper-boundary condition can then be approximated by

$$w(\alpha_R - \alpha_A) D_H = \lambda \theta_A e^{\beta D_H}. \tag{44}$$

Let

$$a = w(\alpha_R - \alpha_A), \qquad x = \frac{\beta \lambda \theta_A}{a}.$$

Equation (44) becomes

$$(-\beta D_H)e^{-\beta D_H} = -x.$$

Proposition 6 (Lambert-W approximation)**.** *If*

$$0 < x \leq e^{-1},$$

the high-complexity solution to Equation (44) *is*

$$D_H \approx -\frac{1}{\beta} W_{-1}\left(-\frac{\beta\lambda\theta_A}{w(\alpha_R - \alpha_A)}\right), \tag{45}$$

where W_{-1} is the lower real branch of the Lambert-W function.

As $x \downarrow 0$,

$$D_H = \frac{1}{\beta}\left[\log\left(\frac{1}{x}\right) + \log\log\left(\frac{1}{x}\right) + o(1)\right]. \tag{46}$$

The leading term is logarithmic in the ratio of labor value to agentic compute cost. Holding λ fixed, a proportional reduction in θ_A therefore produces an approximately additive increase in the upper boundary. Under a hard quota, however, λ is endogenous and may change when θ_A changes. The fixed-λ approximation should therefore be interpreted as an internal-price comparative static, not as a complete quota-equilibrium result.

3.10 Full Agentic Allocation

Full agentic allocation does not require the compute constraint to be strictly slack.

Corollary 7 (Full agentic allocation)**.** *Suppose agentic coding minimizes labor for every task:*

$$L_A(D) \leq \min\{L_C(D), L_R(D)\}$$

for F-almost every D. Assigning every task to A is feasible if and only if

$$\int_{\mathcal{D}} C_A(D)\, dF(D) \leq b. \tag{47}$$

If the inequality is strict, the quota is slack under full agentic allocation. If equality holds, full agentic allocation can coexist with a binding quota.

3.11 Methodology

We construct a cross-organization dataset of publicly accessible issue-tracker records from major open-source and software-development communities, including Apache, Hyperledger, MariaDB, MongoDB, Mojang, Qt, Red Hat, and Sakai. The sample spans infrastructure and database software, enterprise platforms, developer frameworks, education technology, and consumer software. For

each tracker, we collect structured issue-level metadata through publicly accessible Jira interfaces, including issue and project identifiers, issue type, priority, workflow status, resolution, creation and resolution timestamps, labels, components, release-version information, and available story-point values. To limit storage demands and avoid collecting unnecessary user-generated content, our baseline extraction excludes long-form issue descriptions, comments, attachments, worklogs, and complete change histories. The primary analysis sample is restricted to issues with a recorded resolution, which permits measurement of the interval between issue creation and completion. The dataset captures activity observable in the selected public Jira trackers rather than the complete development activity of each organization, as some projects use alternative systems such as GitHub or Bugzilla and tracker availability changes over time.

3.12 Summary Statistics for Public Jira Dataset

We first present the summary statistics from the 2022 Public Jira Dataset.

Table 1: Summary statistics of task complexity and resolution time by project

	Complexity (D)				Resolution (hrs)			
Project	N	Mean	Median	SD	N	Mean	Median	SD
Apache	2683	3.0	3.0	1.8	2683	1638.9	476.6	2913.3
Hyperledger	3155	2.9	3.0	1.8	3155	2095.1	622.4	3303.1
IntelDAOS	6092	4.2	3.0	3.0	6092	2277.7	786.0	3317.0
Jira	3013	2.9	2.0	1.9	3013	3330.1	1201.2	4398.1
JiraEcosystem	1455	2.3	2.0	1.9	1455	2511.3	648.5	3987.4
MongoDB	14654	2.2	2.0	1.7	14654	1525.1	503.1	2668.0
Qt	1837	3.8	3.0	3.5	1837	1724.3	500.2	2879.7
RedHat	23087	3.8	3.0	3.2	23087	1744.9	633.8	2855.4
SecondLife	8	3.0	2.0	2.4	8	2796.2	2791.2	2085.7
Sonatype	1585	1.6	1.0	1.2	1585	2483.4	666.5	3906.6
Spring	3084	3.4	3.0	2.7	3084	1212.5	263.0	2610.5
Pooled	60653	3.2	3.0	2.7	60653	1847.7	597.6	3068.1

Table 2: Summary statistics of α by project

Project	N	Mean	Median	SD
Apache	2683	669.8	167.8	1498.2
Hyperledger	3155	989.8	258.0	4123.4
IntelDAOS	6092	705.9	222.4	1342.1
Jira	3013	1484.6	539.9	2569.1
JiraEcosystem	1455	3166.6	383.3	22169.3
MongoDB	14654	1560.1	246.5	6068.9
Qt	1837	830.0	168.1	2113.8
RedHat	23087	654.9	203.6	1414.2
SecondLife	8	1433.1	1019.0	1361.1
Sonatype	1585	1740.8	518.8	2986.2
Spring	3084	524.9	86.7	1555.4
Pooled	60653	1025.4	227.4	4869.3

and the complexity distribution

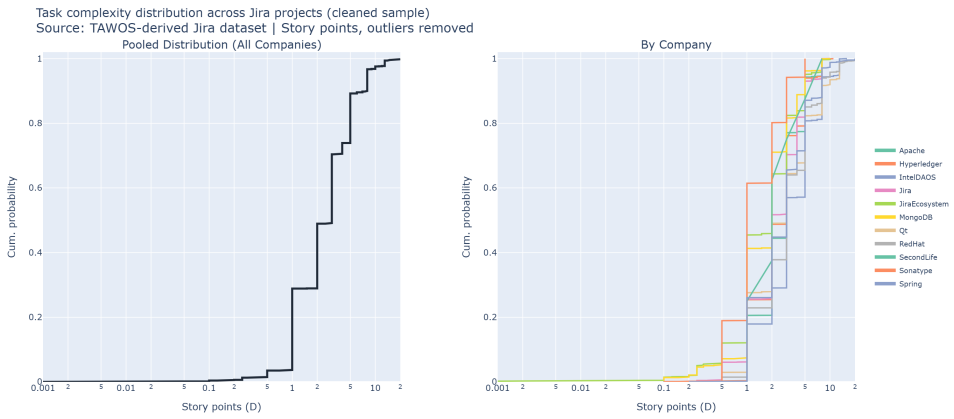


Figure 1: Task Complexity by Company

Here we plot the task complexity distribution for each individual companies in the dataset. Here we can see that the majority of the tasks are between 1 to 5 in terms of complexity and most of the companies have a similar task difficulty distribution, suggesting that our new dataset is likely to capture the overall complexity change overtime even though we have to drop some of the companies from the list.

3.13 Summary Statistics for New Jira Dataset

Now we will present the summary statistics for companies with still active database. Notice here that Sakai and MariaDB are not present becuase none of the issues have story points assigned or estimated story points assigned.

Table 3: Summary statistics of task complexity and resolution time by project, extended duration

	Complexity (D)				Resolution (hrs)			
Project	N	Mean	Median	SD	N	Mean	Median	SD
Apache	4554	2.9	3.0	1.9	4554	5993.0	836.5	9905.5
MongoDb	3535	2.2	2.0	1.7	3535	3101.8	896.9	6626.5
QT	5298	2.9	2.0	2.3	5298	2942.4	335.5	9079.0
redhat	12990	3.7	3.0	3.3	12990	2069.8	617.6	4332.6
Pooled	26377	3.2	3.0	2.8	26377	3060.7	601.8	7112.6

Table 4: Summary statistics of α by project, extended duration

	α (hrs/pt)			
Project	N	Mean	Median	SD
Apache	4554	2930.7	321.2	6034.8
MongoDb	3535	1629.6	447.3	3565.6
QT	5298	1110.8	144.1	4063.0
redhat	12990	1077.9	206.2	2974.8
Pooled	26377	1478.4	229.2	4017.0

and the new complexity distribution

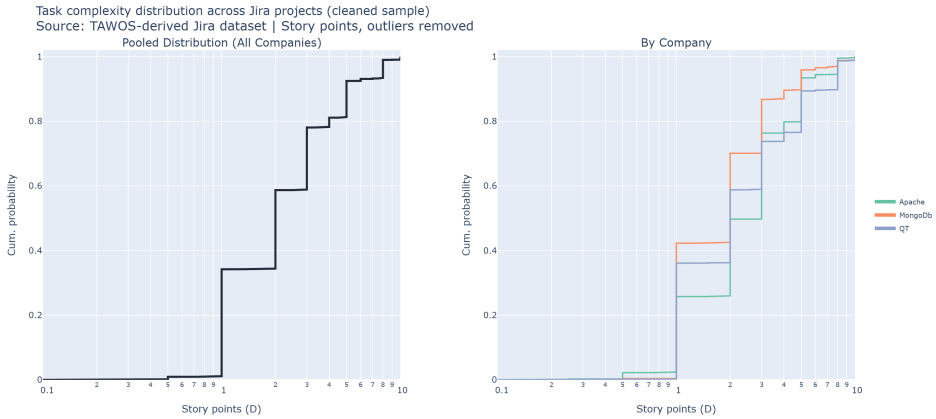


Figure 2: Task Complexity by Company, extended duration

As the tables above shown, the task complexity distribution is still similar to the pre 2022 dataset.

3.14 Task Distribution Approximation

The lognormal distribution provides the best-supported parametric approximation to the empirical task-complexity distribution among the four candidates considered. It achieves both the highest

log-likelihood (-125,777) and the lowest AIC (251,560.1), outperforming the gamma distribution (AIC 253,334.7), Weibull (AIC 256,193.5), and exponential (AIC 263,705.6) by margins well above conventional thresholds for decisive model preference. Kolmogorov-Smirnov tests reject all four candidate distributions at conventional significance levels; given the sample size (n = 60,653), this is expected, since KS test power increases with sample size to the point that even minor, practically immaterial departures from an assumed parametric form are detected as statistically significant. The KS statistics are therefore used only as a secondary, relative diagnostic: the gamma distribution has a marginally lower KS statistic (0.143 vs. 0.148), suggesting it may fit the central mass of the distribution slightly better, while the lognormal's likelihood advantage indicates a better overall fit, particularly in capturing the heavier right tail characteristic of task-complexity data. Because the lognormal dominates on the primary model-selection criterion (AIC) and the theoretical restriction requires only that $F(D)$ have a well-defined right tail — not that it match any particular family exactly — the lognormal is adopted as the baseline calibration distribution for $F(D)$, with the gamma distribution retained as a robustness check given its comparable fit and more convenient conjugacy properties for some of the integrals in Section Now we can test for potential structure breaks using t-test and KS-test.

3.15 Time Series Properties

Table 5: Stationarity Tests for Task-Count Growth and Mean Complexity

	Task-count growth rate			Mean complexity, D		
Project	ADF p	KPSS p	Conclusion	ADF p	KPSS p	Conclusion
Apache	< 0.001	≥ 0.10	Stationary	0.999	≤ 0.01	Nonstationary
Hyperledger	< 0.001	0.067	Stationary	0.455	0.074	Inconclusive
IntelDAOS	< 0.001	≥ 0.10	Stationary	0.634	≤ 0.01	Nonstationary
JiraEcosystem	< 0.001	≥ 0.10	Stationary	< 0.001	≥ 0.10	Stationary
Jira	< 0.001	0.042	Mixed	< 0.001	≥ 0.10	Stationary
MongoDB	0.038	≥ 0.10	Stationary	0.168	0.036	Nonstationary
Qt	0.009	≥ 0.10	Stationary	< 0.001	≥ 0.10	Stationary
RedHat	< 0.001	≥ 0.10	Stationary	0.134	0.077	Inconclusive
Sonatype	< 0.001	≥ 0.10	Stationary	0.230	≤ 0.01	Nonstationary
Spring	0.045	0.042	Mixed	0.006	≥ 0.10	Stationary

Notes: The augmented Dickey–Fuller (ADF) test has a unit-root (nonstationarity) null hypothesis, while the KPSS test has a stationarity null hypothesis. "Stationary" denotes rejection of the ADF unit-root null at the 5% level together with failure to reject the KPSS stationarity null. "Nonstationary" denotes failure to reject the ADF unit-root null together with rejection of the KPSS stationarity null. "Mixed" denotes conflicting ADF and KPSS conclusions; "Inconclusive" denotes failure to reject either test's null hypothesis. KPSS values of ≥ 0.10 and ≤ 0.01 are bounds resulting from the reported p-value range.

Table 5 reports month-level augmented Dickey-Fuller tests. At the 5% significance level, we fail to reject the unit-root null for log task count in every project. Results for mean complexity D are heterogeneous: the null is rejected for Apache, IntelDAOS, JiraEcosystem, Jira, RedHat, and Spring, but not for Hyperledger, MongoDB, Qt, or Sonatype. These results are consistent with the observation that major deadlines and software bugs are cyclical and not distributed equally in a

year.

4 Empirical Calibration and Failure Dynamics

4.1 Reduced Form Calibration

Direct estimation of the modality-specific cost functions is not currently possible because no public dataset simultaneously observes task complexity, human labor, agent trajectories, token usage, and monetary cost for a common task population. We therefore adopt a moment-based reduced-form calibration. The calibration treats the observed task-complexity distribution as given and uses publicly reported aggregate cost moments to discipline the level of the modality-specific cost functions. Parameters governing the curvature of agentic cost remain sensitivity parameters rather than point-identified estimates.

Let D denote task complexity and let $F(D)$ denote its empirical distribution in the task data. Recall that for modality $m \in \{C, R, A\}$, where C denotes code chat, R denotes code reasoning, and A denotes agentic coding, define expected compute cost as

$$C_C(D) = \theta_C D, \qquad C_R(D) = \theta_R D^2, \qquad C_A(D) = \theta_A(\beta)\exp(\beta D).$$

The distinction between the three functional forms reflects increasing degrees of autonomy and context accumulation. In particular, agentic coding involves repeated tool interaction and the accumulation of prior context, making its cost potentially nonlinear in task complexity.

The calibration uses the mean task costs reported by Bai et al.:

$$\bar{C}_C = 0.023, \qquad \bar{C}_R = 0.016, \qquad \bar{C}_A = 1.857.$$

These are aggregate monetary costs per task, rather than conditional costs at a particular value of D. Bai et al. also report average token usage of approximately 3.39 thousand tokens for code chat, 1.19 thousand tokens for code reasoning, and 4.17 million tokens for agentic coding. Agentic coding therefore has a much larger cost level than the other two modalities, and its token usage is dominated by input tokens associated with repeated context processing.

For a given empirical distribution $F(D)$, the linear and quadratic parameters are obtained by moment matching:

$$\theta_C = \frac{\bar{C}_C}{\mathbb{E}_F[D]}, \qquad \theta_R = \frac{\bar{C}_R}{\mathbb{E}_F[D^2]}.$$

The agentic cost parameter cannot be separately identified from the curvature parameter using the mean alone. Instead, for each candidate value of $\beta > 0$, define

$$\theta_A(\beta) = \frac{\bar{C}_A}{\mathbb{E}_F[\exp(\beta D)]}.$$

This construction ensures that every candidate calibration reproduces the same observed mean

agentic cost:

$$\mathbb{E}_F[C_A(D)] = \theta_A(\beta)\mathbb{E}_F[\exp(\beta D)] = \bar{C}_A.$$

Consequently, the baseline results are reported over a calibrated sensitivity set $\mathcal{B}$ rather than for a single arbitrarily chosen value of β.

Table 6: Reduced-form compute-cost calibration

Object	**Baseline specification**	**Calibration value**	**Identification and interpretation**
Task complexity	D, with distribution $F(D)$	$\sigma = 0.807, \mu = 2.385$	From the Jira data. Used to calculate all moments and aggregate costs.
Code chat mean cost	$\bar{C}_C = \mathbb{E}_F[C_C(D)]$	\$0.023 per task	Reported aggregate moment from Bai et al.; used to calibrate θ_C.
Code reasoning mean cost	$\bar{C}_R = \mathbb{E}_F[C_R(D)]$	\$0.016 per task	Reported aggregate moment from Bai et al.; used to calibrate θ_R.
Agentic coding mean cost	$\bar{C}_A = \mathbb{E}_F[C_A(D)]$	\$1.857 per task	Reported aggregate moment from Bai et al.; used to calibrate $\theta_A(\beta)$.
Code chat level	$C_C(D) = \theta_C D$	$\theta_C = \frac{0.023}{\mathbb{E}_F[D]}$	Point calibrated conditional on the chosen complexity scale and empirical distribution.
Code reasoning level	$C_R(D) = \theta_R D^2$	$\theta_R = \frac{0.016}{\mathbb{E}_F[D^2]}$	Point calibrated conditional on the chosen quadratic specification.
Agentic coding level	$C_A(D) = \theta_A(\beta) e^{\beta D}$	$\theta_A(\beta) = \frac{1.857}{\mathbb{E}_F[e^{\beta D}]}$	Not separately identified from β; level is recalibrated for every candidate curvature value.
Agentic curvature	β	$\beta \in \mathcal{B} \subset (0, \infty)$	Bai et al. establish a positive but modest association between human-rated difficulty and token usage.
Agentic token scale	$\mathbb{E}[Q_A]$	Approximately 4.17 million tokens	Aggregate token-use moment reported by Bai et al.
Reasoning token scale	$\mathbb{E}[Q_R]$	Approximately 1.19 thousand tokens	Aggregate token-use moment reported by Bai et al.
Chat token scale	$\mathbb{E}[Q_C]$	Approximately 3.39 thousand tokens	Aggregate token-use moment reported by Bai et al.
Same-task cost risk	$\log \theta_A + \beta D + \varepsilon_A$	Sensitivity parameter σ_A^2	Motivated by cost variation across repeated runs of the same task.
Model/scaffold heterogeneity	$\theta_{A,m} = \theta_A \exp(\delta_m)$	δ_m varies by model	Motivated by persistent cross-model differences in token usage on common success and failure subsets.

Notes: Monetary costs are reported in U.S. dollars per task. The values for mean token usage and mean monetary cost are aggregate moments from Bai et al.'s agentic-coding, code-reasoning, and code-chat comparison. The parameters θ_C, θ_R, and $\theta_A(\beta)$ are calibrated to the empirical distribution $F(D)$, so their numerical values depend on the normalization and support of D. The curvature parameter β is not point identified from the aggregate mean agentic cost and is therefore varied over a prespecified sensitivity set.

4.2 Calibration discipline

The calibration should be interpreted as partial identification. The aggregate cost moments identify the average level of each modality under the selected task distribution, but they do not identify the full conditional relationship between task complexity and cost. In particular, the mean agentic cost provides one restriction on the two parameters θ_A and β. For this reason, the analysis fixes β over a prespecified grid and recalibrates $\theta_A(\beta)$ to preserve the observed mean cost. This approach prevents the model from attributing an arbitrary level difference to curvature and makes clear which results are robust to the unresolved calibration problem.

The empirical evidence supports a positive average relationship between task difficulty and agentic token usage, but it also indicates substantial overlap across difficulty groups. In Bai et al. (2026)., the Kendall rank correlation between human-rated task difficulty and total token usage is 0.32, and the distributions of token usage overlap considerably across difficulty categories. Thus, the data support $\beta > 0$ as a qualitative restriction but do not justify treating a single value of β as directly estimated. The baseline therefore reports threshold and allocation outcomes as functions of β, or equivalently as an identified set over $\beta \in \mathcal{B}$.

4.3 Baseline Compute Scalars (θ)

Cross model telemetry confirms that agentic models ($m = A$) exhibit radically different token profiles than reasoning ($m = R$) or chat ($m = C$) modalities. Average agentic token usage reaches 4.17×10^6 tokens per task, translating to an average task cost of \$1.857 compared to \$0.016 for Code Reasoning. Crucially, the Input/Output token ratio for agentic coding averages 153.85, empirically validating that the primary driver of cost is not novel code generation, but the continuous re-reading of accumulated context.

4.4 Code Effort Shares

Table 7: Descriptive phase shares in agentic coding trajectories

Phase	Description	Share of rounds
Setup	Planning, environment setup, initial reproduction	9.98%
Explore	Code search, file inspection, root-cause analysis	30.37%
Fix	Code edits, debugging, patch refinement	33.53%
Validate	Testing, regression checks, verification	16.59%
Closeout	Final checks, cleanup, summary output	9.53%

Note: Shares are descriptive statistics reported by Bai et al. (2026). They are not assumed to equal shares of monetary cost.

4.5 The Mechanism of Context Window Bloat

Token data decomposed by problem solving phase reveals the volume mechanism behind cost growth. During fix and validate phases, cache-read tokens make up the overwhelming majority of total tokens

consumed per call, frequently exceeding 10^6 tokens per phase. This is consistent with the ReAct-style architecture's core inefficiency: each retry re-sends the full accumulated trajectory (system prompt, prior tool calls, prior errors) as context, and providers cache that static prefix to avoid re-computing it from scratch. (Bai et al. 2026)

Critically, cache reads are billed at a steep discount relative to fresh input tokens, roughly 10% of the standard input rate across Anthropic, OpenAI, and Google (*Prompt caching* 2026), so the exponential cost growth in our model, $C_A(D) = \theta e^{\beta D}$, cannot be attributed to the unit price of tokens. It must instead be attributed to volume, the number of cache-read tokens per call grows linearly with trajectory length, and the number of retry attempts itself grows with task complexity and failure probability. The compounding of these two linear effects, payload size per turn times number of turns, is what produces the effectively super-linear (and, for calibration purposes, well-approximated by exponential) growth in total billed tokens as D increases, even though each individual token is cheap on a cache hit.

Because a arithmetic-series cost structure (cost proportional to $n(n+1)/2$ in the number of retries n is a more accurate description of repeatedly re-sending a growing context window, the exponential form should be understood as a reduced-form approximation to this compounding process near the high-complexity, high-retry regime, rather than a literal description of per-token pricing. Section 6.3 empirically tests whether this approximation holds by comparing fitted β against failure-driven retry counts directly.

5 Numerical Implementation of the Engineer's Problem

This section describes the numerical solution of the engineer's dynamic problem. The purpose of the algorithm is to construct modality-contingent response functions that can subsequently be used in the company's task allocation problem. In particular, for every observed level of task complexity and every available AI modality, the numerical model computes the engineer's optimal human effort, AI reasoning depth, total labor input, compute consumption, and task completion.

The economic structure is nested. First, task complexity determines the amount of progress required to complete a task. Second, conditional on a modality, the engineer chooses how to divide production between human effort and AI-assisted progress. Finally, the company compares the resulting labor, compute, and output consequences across modalities and integrates them over its distribution of tasks.

This approach distinguishes the engineer's privately costly effort from the company's opportunity cost of labor. The engineer chooses effort and AI depth, whereas the company ultimately values the labor capacity, compute consumption, output, and quality consequences generated by those choices.

5.1 Task Complexity and Required Progress

Let $D \in \mathcal{D}$ denote observed task complexity. In the empirical application, $\mathcal{D}$ is the support of the complexity measure in the Jira data. We map observed complexity into a required progress target

according to

$$q(D) = \xi D, \tag{48}$$

where $\xi > 0$ is a scale parameter.

The baseline normalizes $\xi = 1$, so that one unit of measured task complexity corresponds to one unit of required progress. This normalization does not assert that a Jira complexity unit is literally one hour of work. It fixes the units of the latent progress state. Sensitivity analysis can subsequently vary ξ to evaluate whether the model's conclusions depend on the mapping between observed complexity and required progress.

This distinction is important because D is an empirical task index, whereas progress X_t is a state variable in the engineer's optimization problem. Equation (48) provides the explicit bridge between these two objects.

5.2 Economic Interpretation

The conditional problem asks a simple question:

> Given a task of complexity D and a specified workflow modality m, how much human effort and AI reasoning depth would an optimizing engineer use?

Solving this problem generates an indirect production menu for the company. The company does not need to observe the engineer's disutility of work directly. It uses the implied consequences of the engineer's response: labor required, compute consumed, progress produced, and whether the task is completed.

Importantly, the engineer's objective in equation (12) and the company's objective are not identical. The engineer experiences effort disutility and may internalize only part of compute expenditure. The company instead values labor capacity, compute cost, output, and expected remediation costs. Preserving this distinction makes it possible to study organizational misalignment in a later extension.

5.3 Optimality Conditions

For interpretation and numerical validation, define the current-value Hamiltonian

$$\mathcal{H}_m = bX_t - \frac{\eta}{2}e_t^2 - \tau\theta_m \exp(\beta_m k_t) + \lambda_t \left(e_t + \gamma_m k_t^{\varphi_m}\right). \tag{49}$$

The current-value costate satisfies

$$\dot{\lambda}_t = \rho\lambda_t - b, \tag{50}$$

with terminal condition

$$\lambda_T = \kappa \left[q(D) - X_T\right]. \tag{51}$$

When effort is interior, the first-order condition is

$$e_t^* = \frac{\lambda_t}{\eta}. \tag{52}$$

With bounds, the optimal effort is the projection

$$e_t^* = \Pi_{[\underline{e},\overline{e}]}\left(\frac{\lambda_t}{\eta}\right). \tag{53}$$

For an interior AI-depth choice, the optimality condition is

$$\lambda_t \gamma_m \varphi_m k_t^{\varphi_m - 1} = \tau \theta_m \beta_m \exp(\beta_m k_t). \tag{54}$$

The left hand side is the shadow value of the marginal progress generated by additional AI depth. The right-hand side is the marginal compute burden internalized by the engineer. Equation (54) therefore formalizes the central economic tradeoff: deeper AI assistance is chosen until its marginal contribution to task progress equals its marginal internalized compute cost.

The first-order condition need not hold with equality when k_t is at a modality boundary. In those cases, the corresponding Karush-Kuhn-Tucker condition applies. Numerical diagnostics therefore evaluate equation (54) only at interior observations.

5.4 Direct Transcription

The continuous-time problem is solved by direct transcription. Divide the horizon into N intervals of equal length

$$\Delta t = \frac{T}{N}, \qquad t_n = n\Delta t. \tag{55}$$

Controls are approximated as constant within each interval:

$$e_t = e_n, \qquad k_t = k_n, \qquad t \in [t_n, t_{n+1}). \tag{56}$$

Under piecewise-constant controls, the progress state satisfies

$$X_{n+1} = X_n + \Delta t \left(e_n + \gamma_m k_n^{\varphi_m}\right). \tag{57}$$

Because the right-hand side is constant within the interval, this state update is exact conditional on the piecewise-constant control approximation.

The discounted flow payoff is evaluated using midpoint quadrature:

$$\widehat{V}_m^E(D) = \max_{\{e_n,k_n\}_{n=0}^{N-1}} \left\{ \sum_{n=0}^{N-1} \Delta t\, e^{-\rho t_{n+1/2}} \left[b\frac{X_n + X_{n+1}}{2} - \frac{\eta}{2} e_n^2 \right.\right. \\ \left.\left. - \tau\theta_m \exp(\beta_m k_n) \right] + e^{-\rho T} S(X_N; q(D)) \right\}. \quad (58)$$

The resulting finite-dimensional problem contains N effort choices and, when AI depth is not fixed by the modality, N AI-depth choices. The baseline uses a bound constrained quasi-Newton algorithm. Box constraints are imposed directly, which prevents economically infeasible negative effort and ensures that AI depth remains within the range associated with the assigned modality.

5.5 Initialization and Continuation

Nonlinear numerical optimization may depend on starting values. The implementation therefore uses multiple initial conditions for the AI-enabled modalities. These starting points place AI depth near the bottom, midpoint, and top of the modality-specific interval. Initial human effort is then selected to offset the remaining progress requirement:

$$e_n^{(0)} = \Pi_{[\underline{e},\bar{e}]} \left[\frac{q(D)}{T} - \gamma_m \left(k_n^{(0)} \right)^{\varphi_m} \right]. \quad (59)$$

For each (D, m) pair, the numerical solution with the highest engineer utility across starting values is retained. The solution for complexity D is also used as the start for the next observed complexity value within the same modality. This continuation strategy is computationally efficient because optimal policies typically change gradually as the required progress target increases.

5.6 Engineer Response Functions

For each observed D and modality m, the numerical solution produces the following objects:

$$L_m^*(D) = \int_0^T e_m^*(t; D)\, dt, \quad (60)$$

$$C_m^*(D) = \int_0^T \theta_m \exp[\beta_m k_m^*(t; D)]\, dt, \quad (61)$$

$$X_{mT}^*(D) = \int_0^T [e_m^*(t; D) + \gamma_m k_m^*(t; D)^{\varphi_m}]\, dt, \quad (62)$$

$$Y_m^*(D) = \min\{X_{mT}^*(D), q(D)\}. \quad (63)$$

Here, $L_m^*(D)$ is the effective human labor supplied to the task, $C_m^*(D)$ is physical compute consumption, and $Y_m^*(D)$ is completed task output capped at the required target. The numerical

implementation also records the completion ratio,

$$r_m(D) = \min\left\{\frac{X^*_{mT}(D)}{q(D)}, 1\right\}, \tag{64}$$

and the terminal shortfall,

$$h_m(D) = \max\left\{q(D) - X^*_{mT}(D), 0\right\}. \tag{65}$$

These response functions are the connection between the engineer and company problems. Rather than independently assuming labor and compute cost curves at the company level, the model obtains them from the engineer's optimized responses:

$$(D, m) \longmapsto \left[V^E_m(D), L^*_m(D), C^*_m(D), Y^*_m(D), r_m(D)\right]. \tag{66}$$

5.7 Relationship to the Reduced-Form Model

The numerical engineer problem and the earlier reduced-form allocation model operate at different levels. Equation (11) specifies compute consumption as a function of AI reasoning depth k. After optimizing over k, the model produces an induced relationship between compute and observed task complexity:

$$D \longmapsto C^*_m(D). \tag{67}$$

The reduced-form specification

$$C_A(D) = \widetilde{\theta}_A \exp(\widetilde{\beta}_A D) \tag{68}$$

can therefore be interpreted as an approximation to the optimized response function $C^*_A(D)$, not as the primitive technology itself. The curvature parameter β_A in Equation (11) and the induced reduced-form parameter $\widetilde{\beta}_A$ in Equation (68) are conceptually distinct.

This distinction prevents the calibration from using the same parameter twice. The dynamic model specifies how compute needed increases with AI depth, and the reduced form describes how optimized compute rises with observed task complexity.

For diagnostic purposes, the induced response can be summarized by estimating

$$\log C^*_m(D) = a_m + \widetilde{\beta}_m D + u_m(D) \tag{69}$$

over the empirical support of D. The fit of this approximation indicates whether an exponential reduced form is adequate over the range of task complexities actually observed in the data.

5.8 Empirical Support and Computational Efficiency

Let

$$\widehat{\mathcal{D}} = \{D_i : i = 1, \ldots, N_{\text{issues}}\} \tag{70}$$

denote the unique observed complexity values in the Jira sample. The engineer problem is solved once for every pair $(D, m) \in \widehat{\mathcal{D}} \times \mathcal{M}$, rather than separately for all individual issues.

The empirical distribution for company c is

$$\widehat{p}_c(d) = \frac{1}{N_c} \sum_{i \in c} \mathbf{1}\{D_i = d\}. \tag{71}$$

Once the engineer response table has been computed, expected outcomes for any company policy $\pi_c : \widehat{\mathcal{D}} \to \mathcal{M}$ can be evaluated as

$$\mathbb{E}_c[L^*] = \sum_{d \in \widehat{\mathcal{D}}} \widehat{p}_c(d) L^*_{\pi_c(d)}(d), \tag{72}$$

$$\mathbb{E}_c[C^*] = \sum_{d \in \widehat{\mathcal{D}}} \widehat{p}_c(d) C^*_{\pi_c(d)}(d), \tag{73}$$

$$\mathbb{E}_c[Y^*] = \sum_{d \in \widehat{\mathcal{D}}} \widehat{p}_c(d) Y^*_{\pi_c(d)}(d). \tag{74}$$

Thus, the dynamic engineer problem needs to be solved only once for each observed complexity–modality pair. Differences across companies can then be evaluated by changing the empirical weights $\widehat{p}_c(d)$ rather than repeatedly solving the engineer's dynamic program.

5.9 Baseline Numerical Assumptions

Table 8 reports the parameterization used to validate the numerical implementation. These values are not treated as structural estimates. Parameters with natural normalizations establish units and parameters without direct empirical counterparts define a reference scenario and are varied in sensitivity analysis.

Table 8: Engineer-Level Parameterization

Parameter	Interpretation	Baseline	Status
T	Decision horizon	1	Normalization
ρ	Discount rate	0.05	Reference value
b	Flow progress incentive	0	Baseline restriction
η	Effort-cost curvature	1	Utility normalization
τ	Internal compute price	1	Normalization
ξ	Complexity-to-target scale	1	Progress normalization
v	Gross assigned-task value	1	Utility normalization
κ	Terminal target penalty	25	Numerical scenario
$\underline{e}$	Minimum effort	0	Feasibility restriction
$\overline{e}$	Maximum effort	30	Numerical bound
N	Time intervals	60	Numerical choice

Table 9 reports the corresponding modality parameters.

Table 9: Illustrative Modality Parameters

Modality	γ_m	φ_m	θ_m	β_m	$\underline{k}_m$	$\overline{k}_m$
Human/chat (C)	0	0.70	0	0	0	0
Reasoning (R)	0.45	0.70	0.01	0.25	10^{-4}	4
Agentic (A)	0.85	0.70	0.04	0.80	10^{-4}	8

The parameterization imposes three qualitative differences across modalities. First, more autonomous modalities can generate progress more effectively, reflected in a larger γ_m. Second, agentic workflows permit a larger feasible depth range. Third, compute consumption rises more quickly with agentic depth. These orderings encode the model's economic mechanism while the numerical magnitudes are provisional.

In particular, the parameters θ_m and β_m in Table 9 should not be interpreted as estimates from token data. They are primitive parameters defined over the model's latent AI-depth scale. External token evidence can eventually discipline the induced moments $C_m^*(D)$, but it does not directly identify the scale of k.

5.10 Numerical Validation

Four groups of diagnostics are used to evaluate the numerical solution.

Optimization convergence. The optimizer's convergence status is recorded for every (D, m) pair. Multiple initial conditions are compared, and the solution with the highest objective value is retained.

Completion diagnostics. The terminal shortfall $h_m(D)$ and completion ratio $r_m(D)$ are reported. A large shortfall indicates either that the terminal penalty is too weak, that the effort bounds bind, or that the modality cannot feasibly produce the required progress.

Optimality conditions The numerical controls are compared with Equations (53) and (54). Interior first-order-condition residuals should be small. When a control is at a boundary, the one-sided KKT conditions are used instead of the raw gradient.

This distinction is important because a nonzero raw gradient is not evidence of optimizer failure when the solution optimally lies on a constraint boundary.

Grid refinement The engineer problem is re-solved for representative complexity levels using increasingly fine time grids, such as $N \in \{30, 60, 120\}$. For each modality, the analysis compares

$$\widehat{V}_m^E(D), \quad L_m^*(D), \quad C_m^*(D), \quad \text{and} \quad X_{mT}^*(D). \tag{75}$$

The baseline grid is considered adequate when economically relevant outputs change only negligibly under further refinement.

5.11 The Company Problem

The engineer response function is the input to the company problem developed in the next section. Let

$$\mathcal{R}(D, m) = \left[L_m^*(D), C_m^*(D), Y_m^*(D), V_m^E(D)\right]. \tag{76}$$

The company evaluates each modality using the engineer's optimized response rather than selecting effort and AI depth directly. Under a centralized assignment policy, the company selects

$$\pi_c^*(D) \in \arg\max_{m \in \mathcal{M}} \left\{P_m(Y_m^*(D), D) - \mu_{H,c} L_m^*(D) - p_{C,c} C_m^*(D) - Q_m(D)\right\}, \tag{77}$$

where $\mu_{H,c}$ is the shadow price of engineering capacity, $p_{C,c}$ is the firm's compute price, and $Q_m(D)$ is expected quality or remediation cost.

Integrating Equation (77) against $\widehat{F}_c(D)$ produces company level modality shares, labor demand, compute demand, and expected output. The numerical architecture therefore preserves the intended model hierarchy:

$$\text{Engineer optimization} \longrightarrow \text{modality response functions} \longrightarrow \text{company allocation over } \widehat{F}_c(D).$$

6 Numerical Results

6.1 First Best Result

This section evaluates the engineer-level allocation model using the empirical distribution of task complexity. The exercise is semi-structural. The Jira data discipline the distribution of task complexity, while the relative price of compute is calibrated to alternative locations in that distribution. The baseline does not require observations on engineer salaries or an exact mapping from elapsed issue duration to active labor hours. Accordingly, all labor quantities reported below are normalized units rather than hours, and the results should be interpreted as allocation and comparative-static exercises rather than estimates of dollar-valued productivity.

6.2 Calibration and empirical task distribution

The baseline compares code reasoning, denoted by R, with agentic coding, denoted by A. The normalized labor requirements and compute costs are

$$L_R(D) = \alpha_R D, \qquad C_R(D) = \theta_R D^2, \tag{78}$$

$$L_A(D) = \alpha_A D, \qquad C_A(D) = \theta_A \exp(\beta D). \tag{79}$$

Let $p \equiv \lambda / w$ denote the relative shadow price of compute, where λ is the multiplier on the compute constraint and w is the price of effective engineering labor. Because only the relative scale is identified

by the engineer's modality choice, the baseline imposes the normalization

$$\Delta\alpha \equiv \alpha_R - \alpha_A = 1. \tag{80}$$

The engineer chooses the agentic modality if and only if

$$\Psi(D; p) \equiv \Delta\alpha D - p\left[\theta_A \exp(\beta D) - \theta_R D^2\right] \geq 0. \tag{81}$$

Thus, the allocation depends on the relative price of compute rather than on a separately observed salary. This normalization is useful because the available Jira data describe task composition but do not separately identify wages, active labor input, and the shadow value of compute.

The empirical sample contains 60,653 resolved issues with nonmissing task complexity. Mean complexity is 3.234, the median is 3, the 75th percentile is 5, and the 90th percentile is 8. The support extends from 1 to 20. The distribution is therefore concentrated at low integer values and contains substantial probability mass at a small number of complexity scores. This discreteness is important: an empirical quantile identifies a boundary value of D, but assigning every task at that value to one modality can generate an allocation share substantially larger than the nominal quantile.

Table 10: Empirical distribution of task complexity

Group	N	Share	Mean	SD	P25	Median	P75	P90
Aggregate	60,653	1.000	3.234	2.716	1	3	5	8
Bug	17,814	0.294	2.788	2.422	1	2	3	5
Task	16,317	0.269	3.191	2.597	1	3	5	5
Story	11,689	0.193	4.101	3.027	2	3	5	8
Improvement	4,628	0.076	2.369	1.781	1	2	3	5
New Feature	2,250	0.037	2.757	1.845	1	2	4	5

Notes: The table reports the pooled sample and the five most frequent issue types.

6.3 Cutoff calibration

For each scenario $q \in \{0.25, 0.50, 0.75\}$, let

$$D_q = \widehat{F}^{-1}(q) \tag{82}$$

be the corresponding empirical complexity quantile. The relative compute price is selected so that D_q satisfies the pairwise indifference condition,

$$p_q = \frac{\Delta\alpha D_q}{\theta_A \exp(\beta D_q) - \theta_R D_q^2}. \tag{83}$$

The baseline parameters are $\beta = 0.8$, $\theta_A = 0.1436$, and $\theta_R = 0.00156$. These values should be interpreted as a technology calibration under incomplete data. Evidence from agentic coding efficiency motivates a rapidly increasing and dispersed agentic compute cost, but it does not directly identify

the mapping between Jira complexity and realized token use (Bai et al. 2026), more specifically, the Jira story points cannot be easily directly mapped to existing benchmarks. Equation (83) therefore constructs transparent cutoff scenarios rather than claiming to estimate an observed organizational compute price.

Table 11: Baseline cutoff scenarios

Scenario	Target D_q	p_q	Upper cutoff	Agentic share	Expected labor	Expected compute
P25	1	3.144	1.556	0.254	2.980	0.109
P50	3	1.912	3.000	0.690	1.925	0.591
P75	5	0.641	5.000	0.892	1.016	1.912

Notes: The relative compute price p_q is calibrated using equation (83). "Upper cutoff" is the largest numerical root of (81). Agentic shares are realized sample shares under the cost-minimizing rule, not the nominal target quantiles. Labor is normalized by setting $\alpha_R = 1$ and $\alpha_A = 0$. Expected compute is measured in the units implied by θ_A and θ_R. Objectives are not compared across scenarios because p_q differs across rows.

6.4 The intermediate agentic region

The numerical solution reveals that the choice gap is not globally monotone. Under the median scenario, equation (81) has two roots,

$$D_L = 0.368, \qquad D_H = 3.000. \tag{84}$$

The continuous allocation rule is therefore

$$m^*(D) = \begin{cases} R, & D < D_L, \\ A, & D_L \le D \le D_H, \\ R, & D > D_H. \end{cases} \tag{85}$$

This three-region structure follows from the functional forms. At $D = 0$, the agentic modality incurs the positive fixed compute term $p\theta_A$, whereas the labor saving is zero. At intermediate complexity, the linear labor saving can dominate the incremental compute cost. At sufficiently high complexity, the exponential agentic cost dominates both linear labor savings and quadratic reasoning cost. The baseline should therefore be described as an *intermediate agentic region* rather than as a globally monotone allocation rule.

In the observed data, however, the lower crossing lies below the minimum observed complexity, $D_{\min} = 1$. Consequently, equation (85) is observationally equivalent to a single upper cutoff:

$$m^*(D) = A \quad \Longleftrightarrow \quad D \le 3 \tag{86}$$

for every task in the sample. The median-cutoff calibration therefore retains the economically relevant prediction that agentic coding is used for lower- complexity observed tasks, while reasoning is retained above the upper boundary.

The P25 scenario illustrates why both roots must be reported. Calibrating at $D_{0.25} = 1$ yields a second and larger root at $D_H = 1.556$. Since task complexity takes integer values, only observations with $D = 1$ lie in the empirical agentic region, generating an agentic share of 25.4 percent. By contrast, the P50 and P75 scenarios place the high root directly at 3 and 5, respectively.

6.5 Discreteness and realized automation

The median scenario generates an agentic share of 68.98 percent rather than 50 percent. For a discrete empirical distribution, the quantile D_q satisfies

$$\widehat{F}(D_q^-) \leq q \leq \widehat{F}(D_q), \tag{87}$$

but generally does not satisfy $\widehat{F}(D_q) = q$. Because 68.98 percent of sample observations have $D \leq 3$, assigning all boundary tasks to the agentic modality yields

$$\widehat{s}_A = \widehat{F}(3) = 0.6898. \tag{88}$$

Similarly, assigning all tasks with $D \leq 5$ to the agentic modality produces an 89.22 percent agentic share in the P75 scenario.

If an organization must implement an exact target share q, it must ration agentic capacity among tasks at the boundary. Let

$$\pi_q = \Pr(D = D_q), \qquad F_-(D_q) = \Pr(D < D_q). \tag{89}$$

Then assigning a fraction

$$r_q = \frac{q - F_-(D_q)}{\pi_q} \tag{90}$$

of boundary tasks to the agentic modality delivers an exact share q. In a firm, equation (90) need not represent literal randomization. Boundary tasks can instead be ranked using priority, expected success probability, repository familiarity, or review requirements. The baseline retains the deterministic rule and reports realized empirical shares to avoid introducing an unsupported secondary ranking mechanism.

6.6 Normalized labor implications

Under the normalization $\alpha_R = 1$ and $\alpha_A = 0$, an all-reasoning allocation requires

$$\mathbb{E}[L_R(D)] = \mathbb{E}[D] = 3.234 \tag{91}$$

normalized labor units per task. The expected labor requirements in Table 11 imply reductions relative to the all-reasoning benchmark of approximately 7.9 percent in the P25 scenario, 40.5 percent in the P50 scenario, and 68.6 percent in the P75 scenario.

These quantities are not estimates of hours saved. Setting $\alpha_A = 0$ abstracts from review, validation,

and supervisory labor and is useful only for isolating the allocation margin. A robustness exercise can preserve $\Delta\alpha = 1$ while allowing positive labor in both modalities—for example, $\alpha_R = 1.5$ and $\alpha_A = 0.5$. This leaves the cutoff equation and modality assignments unchanged but raises the level of expected labor. The cutoff is therefore determined by marginal labor savings, whereas predicted labor levels depend on the normalization.

6.7 Heterogeneity by issue type

Applying the common median-scenario policy to issue-type-specific empirical distributions produces substantial variation in predicted agentic use. Stories have the highest mean complexity among the five largest categories and the lowest predicted agentic share. Improvements have the lowest mean complexity and the highest predicted share. These differences arise solely from task composition: the technology and relative compute price are held fixed across categories.

Table 12: Baseline allocation by issue type

Issue type	N	Mean D	Median D	Agentic share	Expected labor	Expected compute
Bug	17,814	2.788	2	0.759	1.404	0.610
Task	16,317	3.191	3	0.687	1.875	0.593
Story	11,689	4.101	3	0.586	2.841	0.612
Improvement	4,628	2.369	2	0.775	1.033	0.550
New Feature	2,250	2.757	2	0.746	1.343	0.602

Notes: Every issue type is evaluated under the pooled median calibration, $p = 1.912$ and $D_H = 3$. Expected labor and compute are normalized model quantities. The table reports compositional predictions, not causal effects of issue labels.

Formally, let $F_k(D)$ denote the empirical complexity distribution for issue type k. Under a common policy $m^*(D; p)$, its predicted agentic share is

$$s_{A,k} = \int \mathbf{1}\{m^*(D; p) = A\}\, dF_k(D). \tag{92}$$

When the lower root lies below the empirical support and the upper root is D_H, equation (92) simplifies to

$$s_{A,k} = F_k(D_H). \tag{93}$$

The issue-type results therefore provide a direct empirical illustration of the model's composition channel: categories with more probability mass above the upper agentic boundary have lower predicted agentic adoption.

6.8 Cross-organization composition exercise

As a final descriptive analysis, the median scenario policy is applied to each organization's empirical task distribution. This is not a full company equilibrium. In particular, the exercise holds the compute price, engineer technology, and labor normalization fixed and allows only $F_c(D)$ to vary. The resulting differences should therefore be interpreted as a pure task composition effect.

Table 13: Predicted allocation under company-specific task distributions

Company	N	Mean D	Median D	Agentic share	Expected labor	Expected compute
RedHat	23,087	3.829	3	0.639	2.524	0.626
MongoDB	14,654	2.233	2	0.766	0.969	0.501
IntelDAOS	6,092	4.235	3	0.570	2.995	0.616
Hyperledger	3,155	2.884	3	0.760	1.343	0.693
Spring	3,084	3.427	3	0.655	2.170	0.570
Jira	3,013	2.871	2	0.698	1.569	0.568
Apache	2,683	2.974	3	0.770	1.312	0.764
Qt	1,837	3.831	3	0.643	2.682	0.518
Sonatype	1,585	1.605	1	0.941	0.290	0.534
JiraEcosystem	1,455	2.304	2	0.768	1.020	0.550
SecondLife	8	3.000	2	0.750	1.625	0.562

Notes: All organizations face the pooled median policy, $p = 1.912$ and $D_H = 3$. The exercise isolates variation in empirical task composition $F_c(D)$. SecondLife is retained for completeness but is too small for substantive comparison.

For organization c, the composition based prediction is

$$s_{A,c}^{\text{comp}} = \int \mathbf{1}\{m^*(D;p) = A\}\, dF_c(D). \tag{94}$$

IntelDAOS has the highest mean complexity among the large samples and the lowest predicted agentic share, 57.0 percent. Sonatype has the lowest mean complexity and the highest predicted share, 94.1 percent. RedHat and Qt also have relatively high mean complexity and predicted shares near 64 percent, while MongoDB's lower-complexity distribution implies a predicted share of 76.6 percent. These results are consistent with the model's composition mechanism, but they should not be attributed to differences in organizational AI policies, compute budgets, or engineer productivity.

The company extension developed later endogenizes the organization specific shadow price p_c through a compute budget or capacity constraint. That extension separates two channels:

$$\text{cross-company allocation differences} = \text{task composition differences} + \text{compute scarcity differences}. \tag{95}$$

The present baseline identifies only the first channel.

6.9 Interpretation and limitations

The preliminary results establish three features of the model. First, the exponential agentic cost specification generates an upper boundary beyond which reasoning is preferred. Second, the same specification also generates a lower crossing near zero, the empirically relevant single cutoff policy arises because this lower crossing lies below the observed support in the median and upper scenarios. Third, the discrete empirical distribution maps a common complexity boundary into heterogeneous and discontinuous adoption shares across issue categories and organizations.

The results do not yet establish that the calibrated upper cutoff is optimal for an observed firm,

rather a relative result. The relative compute price is selected to rationalize a scenario rather than estimated from strictly observed compute budgets, and the labor normalization abstracts from agent review and validation. In addition, human rated task difficulty is only weakly aligned with realized agent token use in available agentic coding evidence (Bai et al. 2026). The baseline should therefore be viewed as a proof of concept: it maps an empirical task distribution into allocation predictions under an explicit technology and highlights which parameters must be disciplined in subsequent robustness and company level exercises.

6.10 Firm Level Result

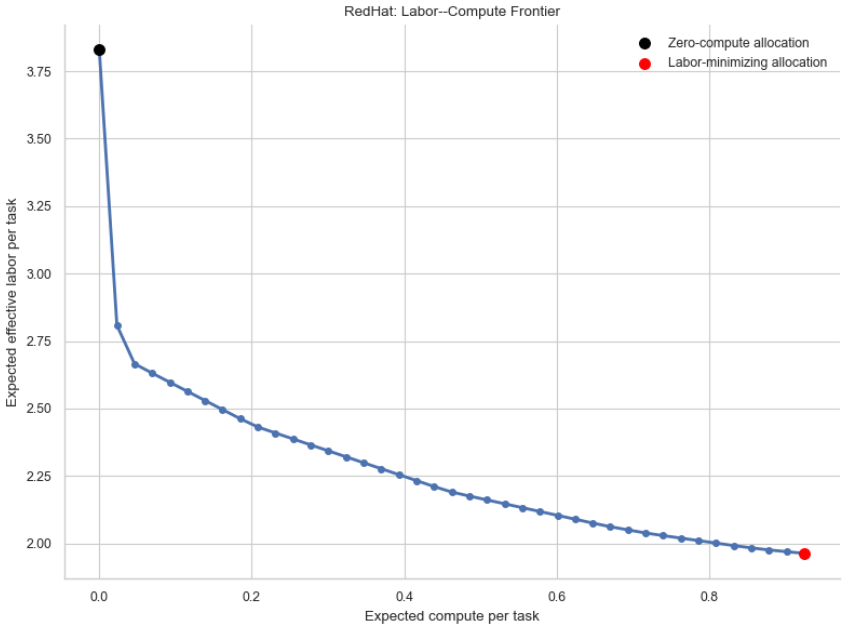


Figure 3: Labor Compute Frontier

Figure 3 reports the associated multiplier on the compute constraint. The marginal value of compute is high near the zero compute allocation and falls sharply after basic reasoning access is introduced. It then declines toward zero as the budget approaches the labor minimizing requirement. This pattern reflects optimal rationing: the firm allocates its first compute units to the task–modality switches producing the greatest labor saving per unit of incremental compute.

The magnitudes are expressed in normalized model units. Consequently, the shadow value should not yet be interpreted as dollars saved per dollar of API expenditure. The exercise identifies the ordering of allocations and the relative scarcity value of compute.

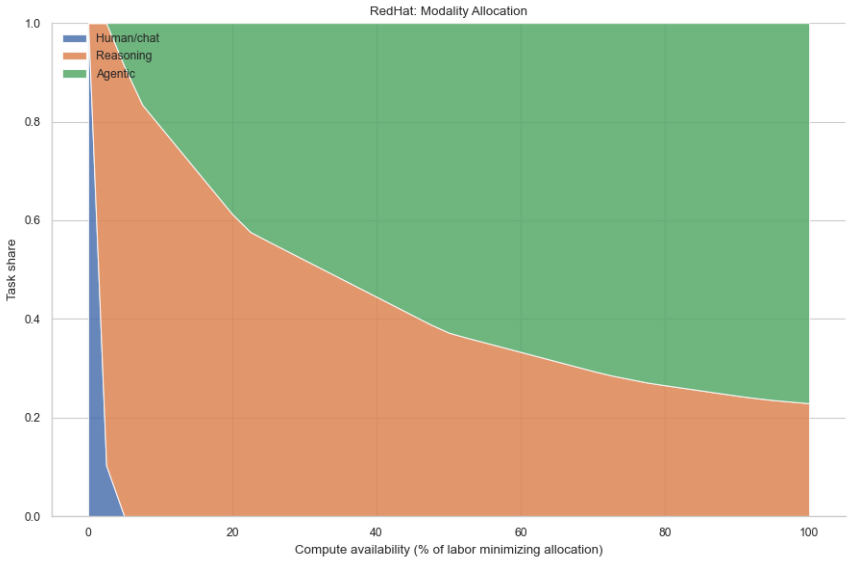


Figure 4: Modality Allocation

Figure 4 reports the optimal allocation of Red Hat's empirical task portfolio as the firm's compute constraint is relaxed. Compute availability is normalized relative to the expected compute required to implement the unconstrained labor-minimizing allocation. A value of zero therefore represents the minimum-compute allocation, while a value of one represents the smallest budget at which the firm can implement its labor-minimizing modality choice at every observed complexity.

The numerical exercise produces three allocation regions. With no available compute, all tasks are assigned to the human/chat modality. Over the first few percentage points of relative compute capacity, the human/chat share falls rapidly and is replaced primarily by reasoning assistance. Once human/chat is eliminated, additional compute shifts tasks progressively from reasoning to agentic workflows.

The sequence indicates that reasoning assistance is an important intermediate technology. The firm first uses scarce compute to obtain relatively inexpensive labor savings through reasoning. More compute and cost intensive agentic workflows expand only after these high return reasoning opportunities have been exploited.

Agentic coding does not become universal at the labor-minimizing endpoint. Approximately three quarters of tasks are assigned to agentic coding, while the remaining tasks continue to use reasoning. Thus, eliminating the compute constraint does not imply complete agentic adoption. For a nontrivial subset of the empirical task distribution, reasoning requires weakly less optimized engineering labor than agentic coding.

7 Conclusion

This paper develops a task based framework for studying the organizational allocation of task based work across chat assisted human effort, reasoning assisted human effort, and agentic workflows. The central economic problem is that AI modalities economize on labor to different degrees while imposing different compute requirements. When compute consumption rises rapidly with task complexity, workflow choice depends jointly on the firm's task distribution, the labor requirements induced by each modality, and the organization's method of governing compute use.

The analytical results distinguish between robust and conditional properties of the allocation. Under a positive compute shadow price, agentic coding is eventually excluded at sufficiently high task complexity whenever its compute requirements grow faster than its labor advantage. This upper-tail result does not, by itself, imply that all tasks below a single threshold should be agentic. Depending on fixed compute requirements and the shapes of the modality cost functions, agentic coding may instead be optimal over an intermediate range of task complexity. A conventional upper threshold policy emerges as a special case when the lower crossing lies below the observed support of task complexity.

The numerical implementation links these theoretical results to organization level decisions. For each task complexity and modality, the engineer problem generates optimized labor, compute, and completion responses. The company then chooses a modality allocation over its empirical task complexity distribution subject to an aggregate compute constraint. This structure separates engineer behavior from organizational resource allocation while maintaining a direct connection between the two levels of the model.

The Red Hat case study shows that the benefits of compute are highly nonlinear. Near the zero-compute allocation, a small increase in capacity creates a large reduction in required engineering labor by allowing tasks to move from human/chat workflows to reasoning assistance. Once these high return substitutions have occurred, additional compute budget shifts tasks from reasoning to agentic workflows, but the marginal labor value of compute declines.

Crucially, agentic coding does not take over the entire task portfolio at the labor minimizing endpoint. Reasoning remains optimal for a meaningful share of tasks even when the compute constraint no longer binds. Thus, greater compute availability expands agentic adoption but does not imply that universal agentic deployment is organizationally efficient. The optimal production system remains heterogeneous, combining reasoning and agentic workflows according to task characteristics.

This finding highlights the distinction between compute scarcity and technological suitability. Some tasks remain non-agentic because compute is scarce, while others remain non-agentic because agentic workflows do not generate enough additional labor savings to justify their use over the cheaper options. Policies that merely expand token budgets may therefore increase agentic adoption without eliminating the economic role of less autonomous AI workflows.

The numerical magnitudes should be interpreted cautiously. Labor and compute are currently measured in normalized model units, and the exercise is not yet a structural estimate of dollar costs, token

consumption, or causal productivity gains. The present contribution is instead to characterize the allocation mechanism and its qualitative implications. Future calibration can map model-compute units into tokens or dollars, incorporate stochastic trajectory costs, and allow engineer productivity and review requirements to vary across tasks and organizations.

Taken together, the results suggest that the relevant organizational question is not whether firms should adopt agentic AI universally. It is how firms should allocate scarce compute and engineering capacity across heterogeneous tasks and AI workflows. The answer depends on the interaction between task complexity, modality-specific labor savings, and the marginal organizational value of compute.

References

Acemoglu, Daron and Pascual Restrepo (May 2019). "Automation and New Tasks: How Technology Displaces and Reinstates Labor". In: *Journal of Economic Perspectives* 33.2, pp. 3–30. DOI: 10.1257/jep.33.2.3. URL: https://www.aeaweb.org/articles?id=10.1257/jep.33.2.3.

Bai, Longju et al. (2026). *How Do AI Agents Spend Your Money? Analyzing and Predicting Token Consumption in Agentic Coding Tasks*. arXiv: 2604.22750 [cs.CL]. URL: https://arxiv.org/abs/2604.22750.

Kwa, Thomas et al. (2026). *Measuring AI Ability to Complete Long Software Tasks*. arXiv: 2503.14499 [cs.AI]. URL: https://arxiv.org/abs/2503.14499.

Ord, Toby (2025). *Is there a half-life for the success rates of AI agents?* arXiv: 2505.05115 [cs.AI]. URL: https://arxiv.org/abs/2505.05115.

Prompt caching (2026). en-US. URL: https://platform.claude.com/docs/en/build-with-claude/prompt-caching (visited on 08/26/2026).

Wang, Xingyao et al. (2025). *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. arXiv: 2407.16741 [cs.SE]. URL: https://arxiv.org/abs/2407.16741.